Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: SCRUM-25 — Dashboard 'The Power Grid'

1. CONTEXTO
Se requiere un dashboard interactivo de simulación de valor con temática cyberpunk para demostrar el ROI de la 'Fábrica de Software Aumentada' de Atlassian. El requerimiento especifica explícitamente que debe ejecutarse localmente abriendo un archivo `index.html` en el navegador (protocolo `file:///`). El stack está predefinido: HTML5, CSS3 puro y Vanilla JavaScript ES6+. No existe backend, no existe servidor, no existe build pipeline.

2. DECISIÓN ARQUITECTÓNICA
Arquitectura: Single-File Application (SFA) con archivo JS plano auxiliar.

Se adoptará una arquitectura de mínima fricción compuesta por exactamente 2 archivos:
- `index.html`: Estructura semántica, estilos CSS embebidos en `<style>` y referencias a scripts.
- `main.js`: TODO el JavaScript de la aplicación en un único archivo plano, sin módulos ES6 (`import`/`export`), sin bundlers, sin dependencias externas.

EVALUACIÓN DE ENTORNO — RESULTADO: LOCAL (file:///)
Por mandato del skill y del requerimiento técnico, queda ESTRICTAMENTE PROHIBIDO el uso de `<script type='module'>`, `import`, `export` o división de lógica en múltiples archivos JS. Todo el JavaScript será un único script plano cargado con `<script src='main.js'>` al final del `<body>`.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS
- **¿Por qué NO React/Vue/Angular?** YAGNI. El requerimiento especifica Vanilla JS. Añadir un framework introduciría una cadena de build (Node.js, npm, webpack/vite) que rompe el criterio de 'abrir en navegador sin servidor'. Costo de mantenimiento: alto. Beneficio: cero para este caso de uso.
- **¿Por qué NO múltiples archivos JS con módulos?** El entorno es `file://`. Los navegadores modernos bloquean `import` en protocolo `file:///` por política CORS. Dividir en módulos haría la aplicación no funcional sin un servidor HTTP. Un único `main.js` plano resuelve el problema sin fricción.
- **¿Por qué NO un servidor local (Live Server, Python http.server)?** El requerimiento dice explícitamente 'sin necesidad de un servidor'. Respetar esa restricción es respetar al usuario final (líder de preventa que abre el archivo en su laptop en una reunión con el cliente).
- **¿Por qué NO CSS en archivo separado?** Se PUEDE separar el CSS en un `styles.css` referenciado con `<link>`, ya que los archivos CSS locales no tienen restricciones CORS. Esto mejora la mantenibilidad sin romper ninguna restricción. Se adoptará esta separación.
- **¿Por qué NO Canvas en Worker?** YAGNI. El dashboard es una demo de preventa, no un motor de juego. `requestAnimationFrame` en el hilo principal a ~60 FPS es más que suficiente para partículas y conexiones. Un OffscreenCanvas Worker añadiría complejidad sin beneficio medible.

4. ESTRUCTURA DE ESTADO Y MÓDULOS
- **`globalState`**: Objeto plano con `modules` (5 booleanos) y flag `overdrive`.
- **`KPI_CONFIG`**: Objeto de configuración con valores base, boosts por módulo, min/max y referencias DOM. Centraliza toda la lógica de negocio.
- **Motor de render**: Un único `requestAnimationFrame` loop que gestiona: partículas (particleCanvas), conexiones con pulsos (connectionCanvas), velocímetro TTM, radar de madurez y transiciones numéricas de KPIs.
- **Separación lógica interna** (sin módulos ES6): El código se organizará en secciones comentadas dentro de

`main.js`: `// === STATE ===`, `// === KPI ENGINE ===`, `// === CANVAS: PARTICLES ===`, `// === CANVAS: CONNECTIONS ===`, `// === CANVAS: CHARTS ===`, `// === UI CONTROLLER ===`, `// === INIT ===`.

5. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1 (Bajo)**: Un único `main.js` de ~600-900 líneas es más difícil de navegar que múltiples módulos. Mitigación: secciones comentadas con separadores visuales claros.
- **Riesgo 2 (Bajo)**: Sin build pipeline, no hay minificación ni tree-shaking. Impacto nulo para una demo local que no viaja por red.
- **Riesgo 3 (Muy Bajo)**: Si en el futuro se requiere servir desde HTTP, la migración a módulos ES6 requeriría refactorizar los `import/export`, pero la lógica de negocio permanecería intacta.
- **Riesgo 4 (Aceptado)**: Las imágenes/iconos de herramientas (GitLab, Datadog, etc.) se representarán con texto/emoji o SVG inline. NO se incluirán archivos binarios `.png/.webp` en la lista de archivos generados.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.js | Estado: APROBADO | Iteraciones: 1
- styles.css | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 43,948
Tokens de salida (output): 22,268
Total tokens: 66,216
Horas-hombre estimadas: ~7.7 horas
Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“3,088 input | !‘1,550 output | 1 llamadas
MICROPLANIFICACION-DISEÑO: !“8,052 input | !‘4,500 output | 3 llamadas
FRONTEND-DESARROLLO: !“8,196 input | !‘16,107 output | 3 llamadas
QA-VALIDACION: !“24,612 input | !‘111 output | 3 llamadas